

Data Analytics Final Project

HOSPITAL BILLING AND TREATMENT ANALYSIS

Project Objective

The final project aims to analyze patient billing data, departments, treatments, room types associated with higher costs and also to study the stay duration, medicine cost, test cost which affect the total bill. Then comparing different admission type with patient bill. Next to identify missing values, duplicate values, inconsistent data and also generating insights that support better hospital financial and operational decisions.

Project Scope

Your project will involve the following key components:

1. Data Loading and Initial Overview

- *Import the dataset using Pandas and provide an overview:*

- Number of rows and columns

```
df.shape      #display number of rows and columns
(1000, 10)
```

- Data types of each column

- **Bill_ID** - Unique identification number for each billing record.
- **Patient_ID** - Unique identification number assigned to each patient.
- **Department** - Hospital department where the patient received treatment.
- **Treatment** - Type of medical treatment provided to the patient.
- **Admission_Type** - Indicates how the patient was admitted, such as Emergency, Urgent, or Elective.
- **Room_Type** - Type/category of room occupied by the patient during the hospital stay.
- **Stay_Days** - Number of days the patient stayed in the hospital.
- **Medicine_Cost** - Amount spent on medicines during the patient's treatment.
- **Test_Cost** - Amount spent on medical/laboratory tests.
- **Total_Bill** - Total amount billed to the patient for the hospital stay and treatment.

- Initial observations (e.g., `head()`, `info()`, `describe()`)

```
df=pd.read_csv("https://raw.githubusercontent.com/MamthaSanthosh/MAIN_PROJECT/refs/heads/main/Raw%20Dataset.csv") #reading csv
df.head()
```

	Bill_ID	Patient_ID	Department	Treatment	Admission_Type	Room_Type	Stay_Days	Medicine_Cost	Test_Cost	Total_Bill
0	B0001	P0366	Cardiology	Blood Test	Planned	Semi-Private	10	8544.0	3182	37426.0
1	B0002	P0383	Pediatrics	Ultrasound	Emergency	General Ward	3	7265.0	10443	23108.0
2	B0003	P0323	Pediatrics	Vaccination	Emergency	General Ward	10	9902.0	2766	25268.0
3	B0004	P0477	General Medicine	Blood Test	Planned	Private	14	7379.0	12164	83243.0
4	B0005	P0099	Cardiology	X-Ray	Emergency	Semi-Private	12	3778.0	3976	38954.0

```
df.tail()
```

	Bill_ID	Patient_ID	Department	Treatment	Admission_Type	Room_Type	Stay_Days	Medicine_Cost	Test_Cost	Total_Bill
995	B0996	P0016	Pediatrics	Physiotherapy	Planned	Semi-Private	4	4569.0	11058	27127.0
996	B0997	P0352	ENT	Ultrasound	Emergency	General Ward	6	6277.0	5425	20702.0
997	B0998	P0160	General Medicine	Physiotherapy	Emergency	Private	8	1360.0	8382	47242.0
998	B0999	P0027	Pediatrics	Consultation	Emergency	General Ward	11	2773.0	4086	21059.0
999	B1000	P0015	General Medicine	ECG	Emergency	Semi-Private	15	1779.0	4735	44814.0

```
df.info() #display dataset information
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Bill_ID         1000 non-null   object
1   Patient_ID      1000 non-null   object
2   Department      1000 non-null   object
3   Treatment       1000 non-null   object
4   Admission_Type  990 non-null    object
5   Room_Type       990 non-null    object
6   Stay_Days       1000 non-null   int64
7   Medicine_Cost   990 non-null    float64
8   Test_Cost       1000 non-null   int64
9   Total_Bill      1000 non-null   float64
```

```
df.describe() #calculating aggregate values
```

	Stay_Days	Medicine_Cost	Test_Cost	Total_Bill
count	1000.000000	990.000000	1000.000000	1000.000000
mean	7.943000	5228.522222	7808.341000	37871.178000
std	4.371597	2822.899978	4143.854877	23875.126005
min	1.000000	516.000000	300.000000	5352.000000
25%	4.000000	2694.250000	4269.500000	23112.000000
50%	8.000000	5375.500000	7661.500000	31429.500000
75%	12.000000	7701.750000	11524.250000	44426.000000
max	15.000000	9990.000000	14983.000000	163994.000000

2. Data Pre-processing

- Perform all necessary cleaning steps such as:

- Handling missing values

```
print("Null values:",df.isnull().sum())

Null values: Bill_ID          0
Patient_ID      0
Department      0
Treatment       0
Admission_Type  10
Room_Type       10
Stay_Days       0
Medicine_Cost   10
Test_Cost       0
Total_Bill      0
dtype: int64
```

- Removing duplicates

```
df.duplicated().sum()

np.int64(0)
```

- Correcting data types

```
df.select_dtypes(include='object').columns    #displaying categorical data

Index(['Bill_ID', 'Patient_ID', 'Department', 'Treatment', 'Admission_Type',
      'Room_Type'],
      dtype='object')
```

- Creating derived columns

```
df.insert(7, 'Room_Rates', None)    #creating a new column in 7th position

df.insert(8, 'Room_Charges', None)  #creating a new column in 8th position

df.columns

Index(['Bill_ID', 'Patient_ID', 'Department', 'Treatment', 'Admission_Type',
      'Room_Type', 'Stay_Days', 'Room_Rates', 'Room_Charges', 'Medicine_Cost',
      'Test_Cost', 'Total_Bill'],
      dtype='object')
```

```
df.insert(11, 'Treatment_Charges', None)
```

	Bill_ID	Patient_ID	Department	Treatment	Admission_Type	Room_Type	Stay_Days	Room_Rates	Room_Charges	Medicine_Cost	Test_Cost	Treatment_Charges	Total_Bill
0	B0001	P0366	Cardiology	Blood_Test	Planned	Semi_Private	10	2500.0	25000.0	8544.0	3182	700.0	37426.0
1	B0002	P0383	Pediatrics	Ultrasound	Emergency	General_Ward	3	1200.0	3600.0	7265.0	10443	1800.0	23108.0
2	B0003	P0323	Pediatrics	Vaccination	Emergency	General_Ward	10	1200.0	12000.0	9902.0	2766	600.0	25268.0
3	B0004	P0477	General_Medicine	Blood_Test	Planned	Private	14	4500.0	63000.0	7379.0	12164	700.0	83243.0
4	B0005	P0099	Cardiology	X_Ray	Emergency	Semi_Private	12	2500.0	30000.0	3778.0	3976	1200.0	38954.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	B0996	P0016	Pediatrics	Physiotherapy	Planned	Semi_Private	4	2500.0	10000.0	4569.0	11058	1500.0	27127.0
996	B0997	P0352	ENT	Ultrasound	Emergency	General_Ward	6	1200.0	7200.0	6277.0	5425	1800.0	20702.0
997	B0998	P0160	General_Medicine	Physiotherapy	Emergency	Private	8	4500.0	36000.0	1360.0	8382	1500.0	47242.0

- Filtering or aggregating data

	Stay_Days	Room_Rates	Room_Charges	Medicine_Cost	Test_Cost	Treatment_Charges	Total_Bill
count	1000.000000	990.000000	990.000000	990.000000	1000.000000	980.000000	1000.000000
mean	7.943000	2792.323232	21990.606061	5228.522222	7808.341000	3126.836735	37871.178000
std	4.371597	2106.896005	23051.235908	2822.899978	4143.854877	3508.498975	23875.126005
min	1.000000	1200.000000	1200.000000	516.000000	300.000000	600.000000	5352.000000
25%	4.000000	1200.000000	8400.000000	2694.250000	4269.500000	800.000000	23112.000000
50%	8.000000	2500.000000	14400.000000	5375.500000	7661.500000	1500.000000	31429.500000
75%	12.000000	4500.000000	27500.000000	7701.750000	11524.250000	4500.000000	44426.000000
max	15.000000	10000.000000	150000.000000	9990.000000	14983.000000	12000.000000	163994.000000

3. Exploratory Data Analysis (EDA)

- Conduct descriptive and exploratory analysis to uncover patterns and trends:
 - Univariate, bivariate, and multivariate analysis

UNIVARIANT

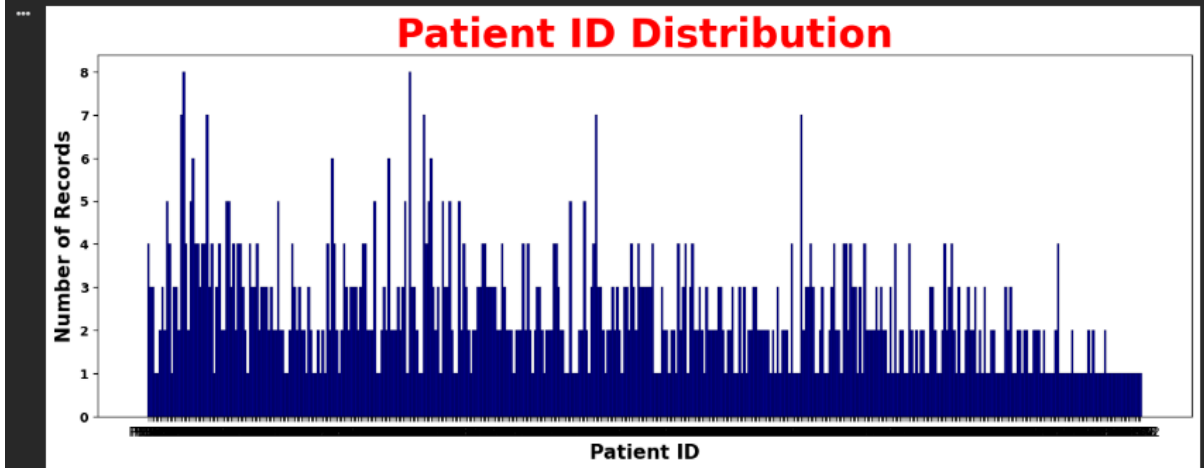
HISTOGRAM

```
df['Patient_ID'].describe()
```

Patient_ID	
count	1000
unique	422
top	P0127
freq	8

dtype: object

```
plt.figure(figsize=(15, 5))
sb.histplot(data=df, x='Patient_ID', facecolor='darkblue', alpha=0.35)
plt.title('Patient ID Distribution', color='red', size=30, fontweight='bold')
plt.xlabel('Patient ID', size=15, fontweight='bold')
plt.ylabel('Number of Records', size=15, fontweight='bold')
plt.yticks(fontweight='bold')
plt.show()
```



```
df['Department'].value_counts()
```

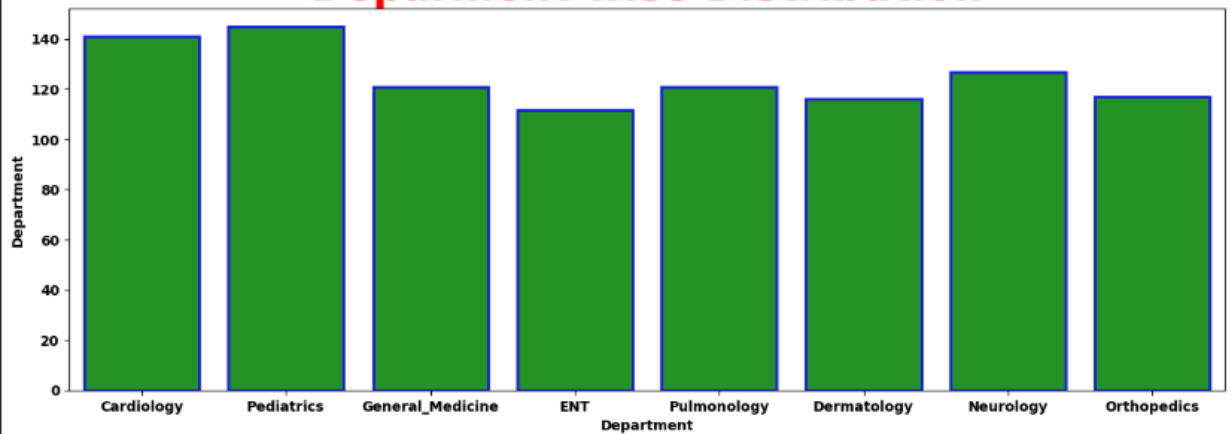
	count
Department	
Pediatrics	145
Cardiology	141
Neurology	127
General_Medicine	121
Pulmonology	121
Orthopedics	117
Dermatology	116
ENT	112

```
dtype: int64
```

```
plt.figure(figsize=(15, 5))
sb.countplot(x='Department', data=df,facecolor='green',alpha=0.85,edgecolor='blue',linewidth=2)
plt.title('Department wise Distribution',color='red',size=30,fontweight='bold')

plt.xticks(rotation=0,size=10,fontweight='bold')
plt.yticks(fontweight='bold')
plt.xlabel('Department',fontweight='bold')
plt.ylabel('Department',fontweight='bold')
plt.show()
```

Department wise Distribution

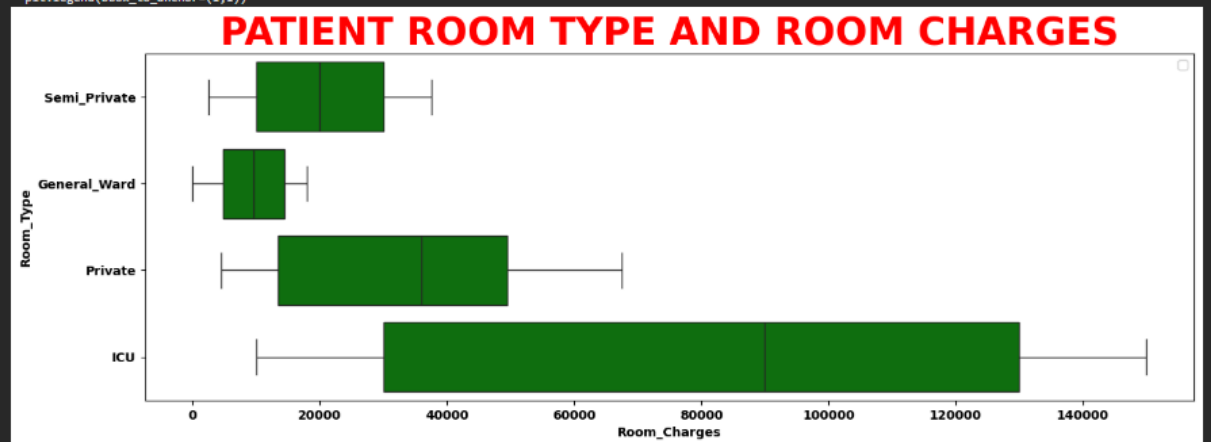


BIVARIANT

BOXPLOT

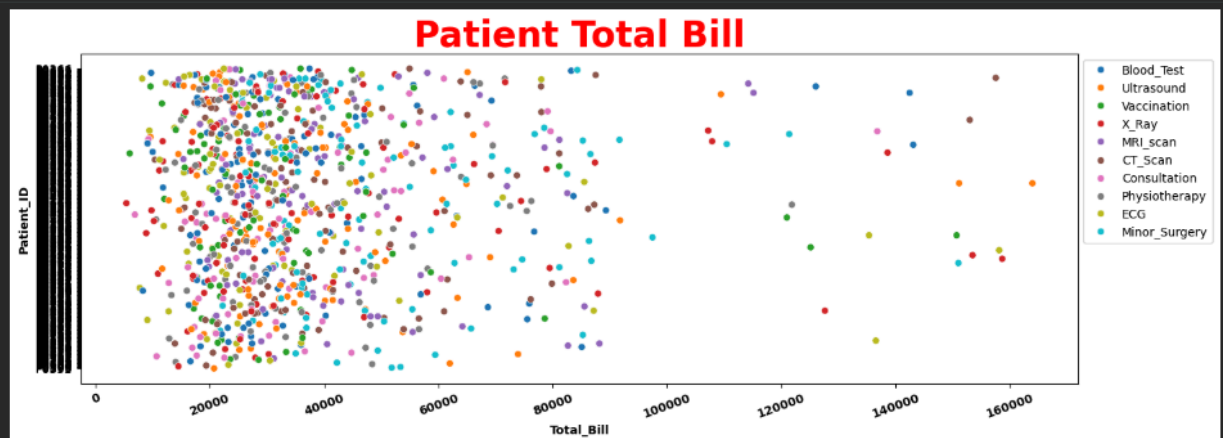
```
plt.figure(figsize=(15,5))
sb.boxplot(
    data=df,
    x="Room_Charges",
    y="Room_Type",
    orient="h",
    color='green'
)
plt.title("PATIENT ROOM TYPE AND ROOM CHARGES",fontweight='bold',color='red',size=30)
plt.legend(bbox_to_anchor=(1,1))
plt.xlabel("Room_Charges",fontweight='bold')
plt.ylabel("Room_Type",fontweight='bold')
plt.xticks(fontweight='bold')
plt.yticks(fontweight='bold')
plt.show()
```

/tmp/ipykernel_1166/153627321.py:10: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend is created from a figure.



SCATTER PLOT

```
plt.figure(figsize=(15,5))
sb.scatterplot(y="Patient_ID", x="Total_Bill", data=df, hue="Treatment")
plt.title("Patient Total Bill",size=30,fontweight='bold',color='red')
plt.xticks(rotation=20,fontweight='bold')
plt.xlabel("Total_Bill",fontweight='bold')
plt.ylabel("Patient_ID",fontweight='bold')
plt.legend(bbox_to_anchor=(1,1))
plt.show()
```



BARPLOT

```
plt.figure(figsize=(15,5))
sb.barplot(x='Department', y='Test_Cost', data=df ,color='darkgreen',edgecolor='red',linewidth=2)
plt.title('TEST COST BY DEPARTMENT',fontweight='bold',size=30,color='red')
plt.xticks(rotation=20,fontweight='bold')
plt.yticks(rotation=20,fontweight='bold')
plt.xlabel('Department',fontweight='bold')
plt.ylabel('Test_Cost',fontweight='bold')
plt.show()
```



CORRELATION HEATMAP

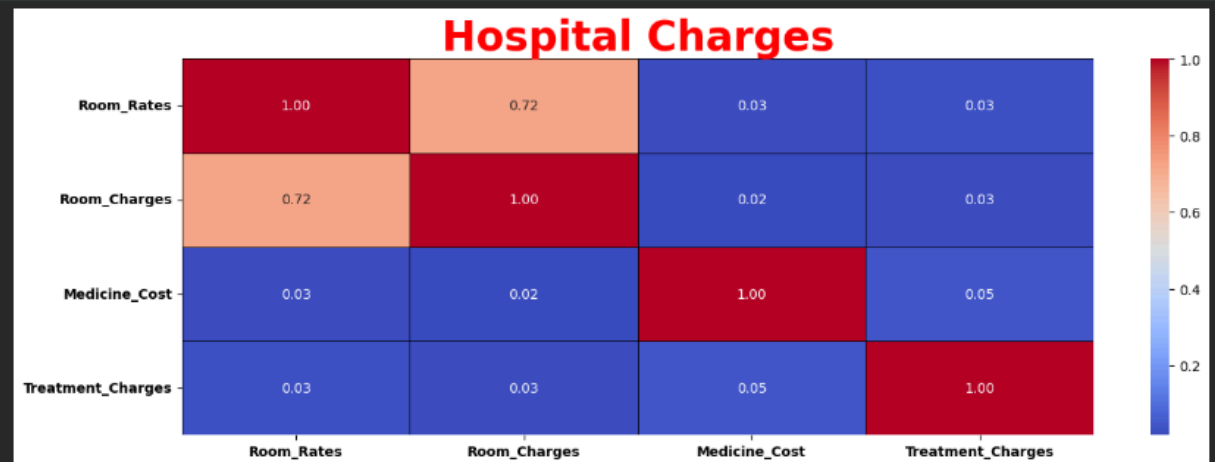
```
numeric_cols = [  
    'Room_Rates',  
    'Room_Charges',  
    'Medicine_Cost',  
    'Treatment_Charges'  
]
```

```
correlation = df[numeric_cols].corr()    #values lies from -1 to 1  
print(correlation)
```

	Room_Rates	Room_Charges	Medicine_Cost	Treatment_Charges
Room_Rates	1.000000	0.723035	0.025413	0.034721
Room_Charges	0.723035	1.000000	0.018802	0.025324
Medicine_Cost	0.025413	0.018802	1.000000	0.048415
Treatment_Charges	0.034721	0.025324	0.048415	1.000000

```
plt.figure(figsize=(15, 5))  
sb.heatmap(  
    correlation,  
    annot=True,  
    cmap='coolwarm',  
    fmt='.2f',  
    linewidths=0.5,  
    linecolor='black'  
)
```

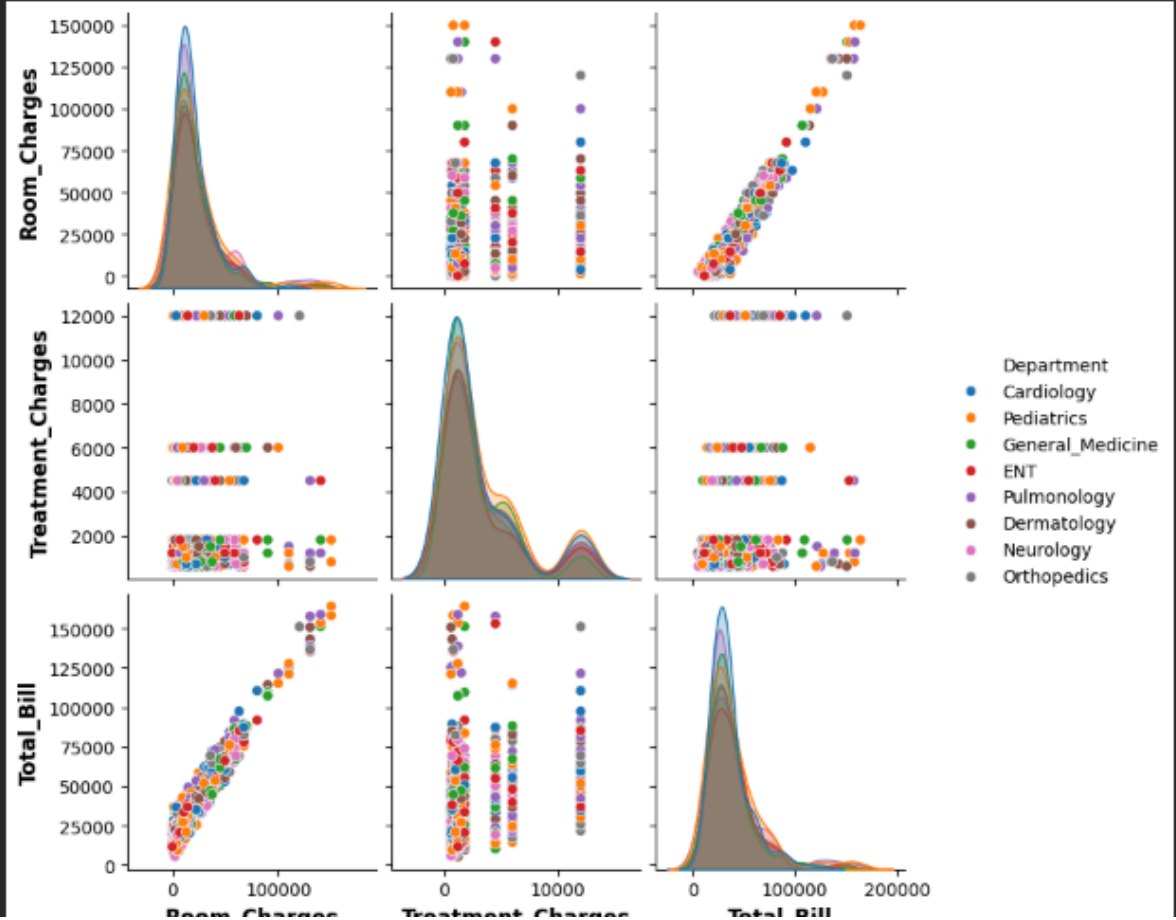
```
plt.title('Hospital Charges',fontweight='bold',size=30,color='red')  
plt.xticks(fontweight='bold')  
plt.yticks(fontweight='bold')  
plt.show()
```



PAIRPLOT

```
plt.figure(figsize=(15,10))
g=sb.pairplot(
    df,
    vars=['Room_Charges', 'Treatment_Charges', 'Total_Bill'],
    hue='Department'
)
#g.fig.suptitle('RELATIONSHIP BETWEEN ROOM_CHARGES,TREATMENT_CHARGES AND TOTAL_BILL',fontweight='bold',fontsize=15,color='r')
for x in g.axes.flat:
    #sets x and y label size,weight
    if x is not None:
        x.set_xlabel(
            x.get_xlabel(),
            fontsize=12,
            fontweight='bold'
        )
        x.set_ylabel(
            x.get_ylabel(),
            fontsize=12,
            fontweight='bold'
        )
plt.show()
```

<Figure size 1500x1000 with 0 Axes>



- Use groupby, pivot tables, and correlation analysis

GROUPED ANALYSIS

```
df.groupby('Department').size() #fin
```

0	
Department	
Cardiology	141
Dermatology	116
ENT	112
General_Medicine	121
Neurology	127
Orthopedics	117
Pediatrics	145
Pulmonology	121

dtype: int64

```
department_count = df.groupby('Department
```

```
print(department_count)
```

	Department	Patient_Count
0	Cardiology	141
1	Dermatology	116
2	ENT	112
3	General_Medicine	121
4	Neurology	127
5	Orthopedics	117
6	Pediatrics	145
7	Pulmonology	121

```
df.groupby('Department')['Total_Bill'].m
```

Total_Bill	
Department	
Cardiology	38075.482270
Dermatology	38642.689855
ENT	38250.151786
General_Medicine	35853.016529
Neurology	35676.291339
Orthopedics	37409.854701
Pediatrics	40534.241379
Pulmonology	40449.950413

dtype: float64

```

department_bill = df.groupby('Department')['Total_Bill'].mean().
print(department_bill)

```

	Department	Total_Bill
0	Cardiology	36075.482270
1	Dermatology	38642.689655
2	ENT	38250.151786
3	General_Medicine	35853.016529
4	Neurology	35676.291339
5	Orthopedics	37409.854701
6	Pediatrics	40534.241379
7	Pulmonology	40449.950413

```

department_total = df.groupby('Department')['Total_Bill'].sum().
print(department_total)

```

	Department	Total_Bill
0	Cardiology	5086643.0
1	Dermatology	4482552.0
2	ENT	4284017.0
3	General_Medicine	4338215.0
4	Neurology	4530889.0
5	Orthopedics	4376953.0
6	Pediatrics	5877465.0
7	Pulmonology	4894444.0

```

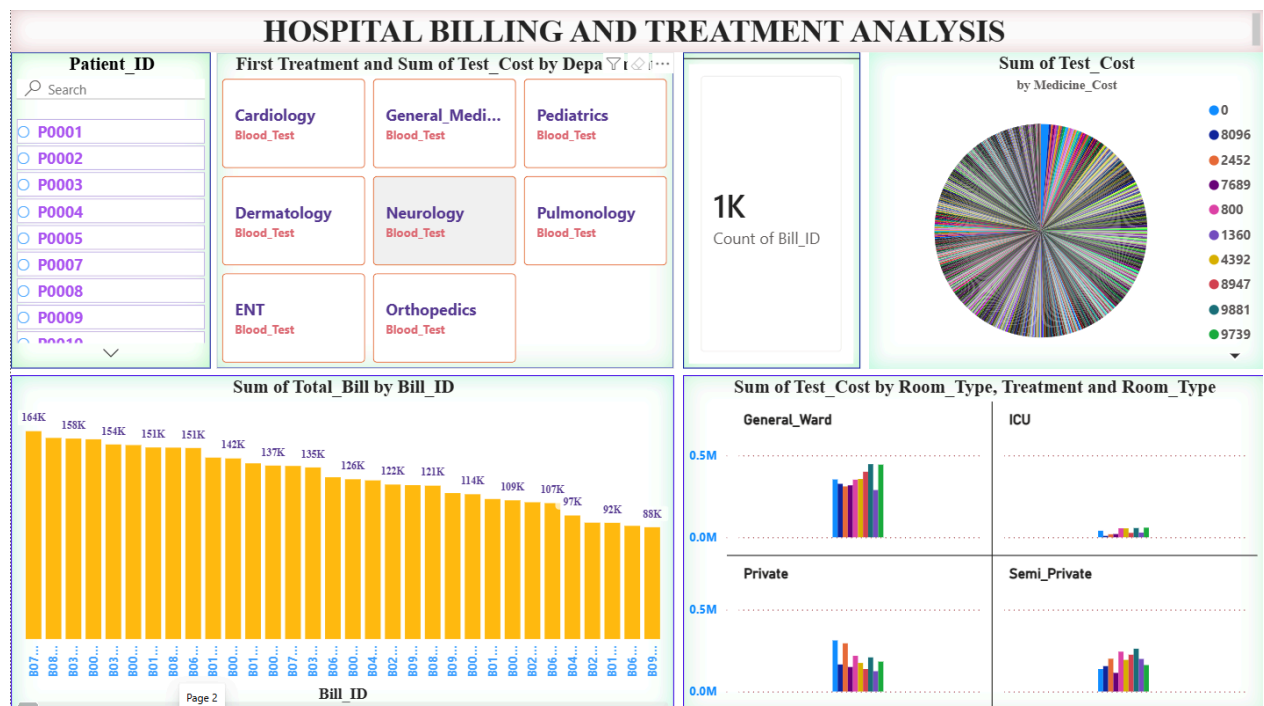
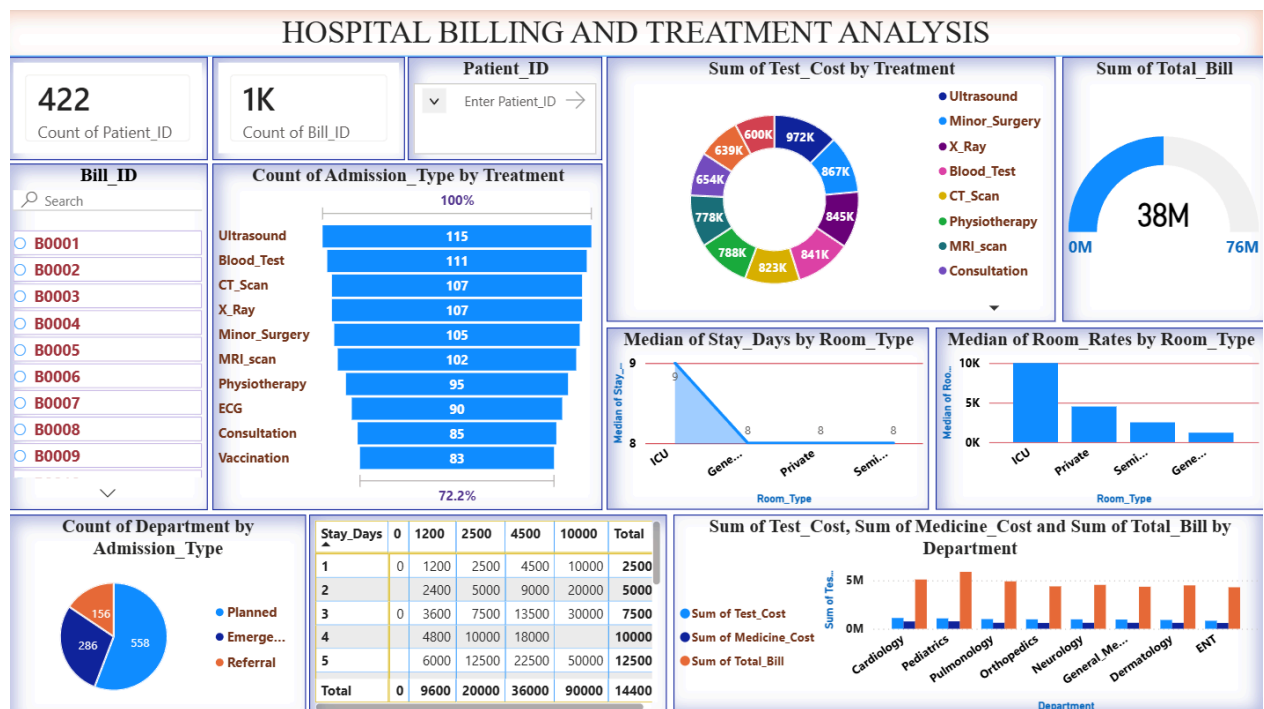
grouped_analysis = df.groupby('Department').agg(
    Patient_Count=('Patient_ID', 'count'),
    Average_Room_Charges=('Room_Charges', 'mean'),
    Average_Treatment_Charges=('Treatment_Charges', 'mean'),
    Total_Revenue=('Total_Bill', 'sum'),
    Average_Bill=('Total_Bill', 'mean')
).reset_index()
print(grouped_analysis)

```

	Department	Patient_Count	Average_Room_Charges	Average_Treatment_Charges	Total_Revenue	Average_Bill
0	Cardiology	141	19777.304965	3040.425532	5086643.0	36075.482270
1	Dermatology	116	22455.172414	3100.000000	4482552.0	38642.689655
2	ENT	112	22477.678571	3075.892857	4284017.0	38250.151786
3	General_Medicine	121	20147.933884	2756.198347	4338215.0	35853.016529
4	Neurology	127	20151.181102	3062.204724	4530889.0	35676.291339
5	Orthopedics	117	20866.666667	3142.735043	4376953.0	37409.854701
6	Pediatrics	145	24375.172414	3473.793103	5877465.0	40534.241379
7	Pulmonology	121	23858.677686	3254.545455	4894444.0	40449.950413

- Include statistical summaries to support findings

4. Visualizations



5. Insight Generation and Report

Summarize key insights from your analysis:

- Patient workload is not evenly distributed across departments.
- Pediatrics and Cardiology account for relatively higher patient volumes, which

can be considered when reviewing staffing and resource requirements.

- Neurology has a relatively high billing level per patient even though it is not the department with the highest number of records.
- Ultrasound and CT Scan records have the highest average bills among the listed treatments, while Endoscopy has the lowest average bill.
- Emergency admissions have a higher average bill than planned.
- ICU records have a substantially higher average bill than the other room categories.
- The correlation between Stay_Days and Total_Bill is approximately 0.63
- Test cost has a positive but relatively weaker linear association with total billing compared with Stay_Days.
- Medicine cost has a relatively weak positive linear association with total billing in this dataset.
- These high-value records should be treated as billing outliers/high-value cases for further investigation. Their department, treatment, room type, length of stay, medicine cost, and test cost can be examined to understand the components associated with these unusually high bills.

Future Enhancement

Future analysis can extend the current hospital billing study from understanding past patterns to predicting future costs and patient demand. Machine-learning models, time-series forecasting, anomaly detection, patient segmentation, and advanced Power BI dashboards can provide deeper insights. These techniques can help hospital management improve cost control, resource planning, billing monitoring, and data-driven decision-making.

Conclusion

From the analysis, we identified meaningful relationships and patterns in the dataset. The visualizations and insights highlight the main factors influencing the outcomes and provide a strong foundation for business decisions.